

had in its lineage. In wide transformations that use hash partitioning, such as `aggregateByKey`, the optional number of partitions parameter is used as an argument to the hash partitioner. See [“Number and Size of Partitions”](#) for information and advice about setting the `spark.default.parallelism` value and choosing the number of partitions.

Range Partitioning

Range partitioning assigns records whose keys are in the same range to a given partition. Range partitioning is required for sorting since it ensures that by sorting records within a given partition, the entire RDD will be sorted. The range partitioner first determines the range bounds for each partition by sampling; optimizing for an equal distribution of records across partitions. Then, each record in the RDD will be shuffled to the partition whose range bounds include the key. Highly unbalanced data (i.e., lots of values for some keys and not others, and if the distribution of the keys is not uniform) makes the sampling less accurate—and, as we have discussed, uneven partitioning may cause downstream tasks to be slower than others, causing “straggler” tasks. If there are too many duplicate keys for all the records associated with one key to fit on one executor, then range partitioning, like hash partitioning, may cause memory errors. Performance problems associated with sorting are usually caused by these problems with the range partitioning step.

Creating a `RangePartitioner` with Spark requires not only a number of partitions, but also the actual RDD to sample. The RDD must be a tuple and the keys must have an ordering defined.

Sampling actually requires partially evaluating the RDD, causing a break

in the execution graph. Thus range partitioning is actually both a transformation and an action. The cost of sampling means that, in general, range partitioning is more expensive than hash partitioning. The requirement that keys be ordered means that range partitioning cannot be done on all RDDs of tuples. Therefore, key/value operations (such as aggregations), which require records with each key to be on the same machine but not ordered in a particular way, use a `HashPartitioner` as the default. However, these methods can also be performed with a custom partitioner or range partitioner.

Custom Partitioning

To define a unique function for partitioning the data other than by the key's hash-value or ordering, Spark allows the user to define a custom partitioner. In order to define a partitioner, you must implement the following methods:

`numPartitions`

A method that returns an integer number of partitions. Expect that this number is greater than zero.

`getPartition`

A method that takes a key (of the same type as the RDD being partitioned) and returns an integer representing the index of the partition that specifies where records with that key belong. The integer must be between zero and the number of partitions (defined in the `numPartitions` method).

`equals`

An (optional) method to define equality between partitioners. The equality method for a `HashPartitioner` returns true if the number of partitions are equal. The range partitioner does so only if the range

bounds are equal. The equality of partitioners can be particularly important for joins and cogrouping, because in some instances if an RDD is already partitioned according to a partitioner, Spark is smart enough not to shuffle again with the same partitioner. We will discuss this in more detail in [“Leveraging Co-Located and Co-Partitioned RDDs”](#).

hashCode

This method is required only if the equals method has been overridden. The hashCode of a `HashPartitioner` is simply its number of partitions. The hashCode of a `RangePartitioner` is a hash function derived from the range bounds.

Preserving Partitioning Information Across Transformations

Some wide transformations change the partitioning of an RDD, as will be shown in [Table 6-4](#). Spark remembers this information by updating the partitioner property of the RDD. When doing a series of transformations, it is important to understand how an RDD is partitioned, because in some instances we can use this information to avoid doing future shuffles.

USING NARROW TRANSFORMATIONS THAT PRESERVE PARTITIONING

Some narrow transformations, such as `mapValues`, preserve the partitioning of an RDD if it exists. Unless a transformation is known to only change the value part of the key/value pair in Spark, the resulting RDD will not have a known partitioner (even if the partitioning has not changed). It’s important to note that common transformations like `map` and `flatMap` can change the key, so even if your function does not change the key, the resulting RDD will not have a known partitioner. Instead, if we don’t want to modify the keys, we can call the `mapValues`

function (defined only on pair RDDs) because it keeps the keys, and therefore the partitioner, exactly the same. The `mapPartitions` function will also preserve the partition if the `preservesPartitioning` flag is set to true. Assuming we have some RDD data of type `RDD[(Double, Int)]`, we could write the test in [Example 6-7](#) to illustrate this property.

Example 6-7. Maintaining partitioning information with `mapValues`

```
val sortedData = data.sortByKey()
val mapValues: RDD[(Double, String)] =
sortedData.mapValues(_.toString)
assert(mapValues.partitioner.isDefined,
    "Using Map Values preserves partitioning")

val map = sortedData.map( pair => (pair._1, pair._2.toString))
assert(map.partitioner.isEmpty, "Using map does not preserve
partitioning")
```



Leveraging Co-Located and Co-Partitioned RDDs

Co-located RDDs are RDDs with the same partitioner that reside in the same physical location in memory. Co-location is important because all of the `CoGroupedRDD` functions—a category which includes the `cogroup` operations and all of the join operations—require the RDDs being grouped to have all of their partitions co-located. RDDs can only be combined without any network transfer if they have the same partitioner and if each of the corresponding partitions in-memory are on the same executor. Partitions will be in-memory on the same executor if they were partitioned in the lineage associated with the same job.

Co-partitioning is related to but distinct from partition co-location. We say that multiple RDDs are *co-partitioned* if they are partitioned by the same known partitioner. We say that partitions are *co-located* if they are both

loaded into memory on the same machine. RDDs are only guaranteed to be co-located if they are put into memory by the same job and the same partitioner: if one action contains the partitioning of both RDDs in its lineage. RDDs will be co-partitioned if their partitioner objects are equal, but if the corresponding partitions for each RDD are not in the same physical location. Recall that “same partitioner” means the partitioner objects are equal according to the equality function defined in the partitioner class.

In [Example 6-8](#), both `rddA` and `rddB` will be co-located.

Example 6-8. An example of co-located RDDs

```
val rddA = a.partitionBy(partitionerX)
rddA.cache()
val rddB = b.partitionBy(partitionerY)
rddB.cache()
val rddC = a.cogroup(b)
rddC.count()
```

Before Spark evaluates `RDDC.count()`, neither RDD is actually loaded into memory due to Spark’s lazily evaluated nature. When Spark launches the associated `RDDC.count()` job, both RDDs are pulled into memory since their lineages are merged by the `cogroup` operation. In this case the join won’t cause any network traffic because both RDDs are loaded into memory in the same location.

In contrast, if we were to call an action on `rddA` and `rddB` before the action on the co-grouped RDD, the `cogroup` may cause some network traffic since the RDDs are not co-located ([Example 6-9](#)).

Example 6-9. RDDs co-partitioned but not co-located

```
val rddA = a.partitionBy(partitionerX)
rddA.cache()
val rddB = b.partitionBy(partitionerY)
```

```
rddB.cache()
val rddC = a.cogroup(b)
rddA.count()
rddB.count()
rddC.count()
```

In this case, `rddA` and `rddB` are loaded into memory from different actions. They are co-partitioned, but there is no guarantee that their partitions will all be co-located. Thus, although the repartition calls prevent the join operator from triggering shuffles in both RDDs, there may still be some network traffic to line up the partitions and load both RDDs into memory. Although the design of your program may require calling actions in this order, it is often worth thinking about the lineage of an RDD before calling an action on it, so as to minimize network traffic.

Dictionary of Mapping and Partitioning Functions

PairRDDFunctions

The performance of the different mapping and partitioning functions available on PairRDDs is shown in [Table 6-4](#).

Table 6-4. Dictionary of mapping and sampling operations for RDDs or key/value pairs

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitions
	Apply a mapping			Slow when the	In co map pres parti the c in fu oper the i has i parti

mapValues	function to each value of a pair RDD without changing the key.	None	Almost never	mapping operation is very complicated or expands the size of each record.	output will sample parts; you perform map just; it is always beneficial so.
flatMapValues	Perform the flatMap function on just the values of a key/value RDD. See our discussion of the flatMap paradigm near the end of “Goldilocks Version 0: Iterative Solution” .	None	Unlikely unless the function applied to each value is very expensive or the result iterator is very large, causing a dramatic expansion of the number of records for each key.	Slow when the mapping routine is very complicated, creates many new objects, or dramatically increases the number of records (such as tokenization).	Preserve parts; associated with the i; How distributed; duplicate values; character manipulation; duplicate values; creation may slow down shuffle.
keys	Return an RDD of just the keys (not distinct).	None	Almost never	Essentially free	Preserve parts
values	Return an RDD of just the values.	None	Almost never	Essentially free	Does preserve parts; Future transactions; will shuffle them; sample.

					partitions of the input RDD
sampleByKey	Given a map from the keys to the percent of each key to sample, returns a stratified sample of the input RDD. The function is implemented with mapPartitions and uses a random number to determine whether each record will be kept. Thus, the size of the resulting sample may not exactly correspond to the percentage specified.	None	Almost never, unless the key map is too large to be broadcast to one of the worker nodes.	Same as mapPartitions, the function completes one pass through the data and does not require a shuffle.	Preserves the input RDD's partitions.

In addition to sampling and mapping operations specific to PairRDDs, [Table 6-5](#) shows the partitioning functions.

Table 6-5. Partitioning functions

Function	Purpose	Key restriction	Runs out of memory when	Slow when	Output partitioner
partitionBy	Takes a partitioner object and partitions the RDD accordingly. The partitioner object defines target partitions.	Depends on which partitioner is used. See “Using the Spark Partitioner Object” .	When there are many duplicate values for each key, regardless of	Always causes a shuffle. Range partitioners are generally slower than hash partitioners since they require the	Partitioned according to the partition argument.

index for
records
based on
key.

partitioner.

data to be
partially
evaluated
in order to
sample it.

The `PairRDDFunctions` class also includes all of the multiple RDD operations including `join` and `cogroup`, which we covered in detail in [Chapter 4](#).

Dictionary of OrderedRDDOperations

In addition to special functions for RDDs of key/value pairs, RDDs with an ordering on the key have even more functions, summarized in [Table 6-6](#).

Table 6-6. Dictionary of operations in the `OrderedRDDFunctions` class

Function	Purpose	Key restriction	Runs out of memory when	Slow when
<code>sortByKey</code>	Return an RDD sorted by the key.	Key must have implicit ordering.	When data is very unevenly distributed both in terms of repeated keys and distribution of keys. In particular, if the data associated with one key can't fit in memory.	Slows down with many keys. If the keys are not distributed, the data will be partitioned unevenly, which causes straggler tasks. If the few nonpure transformations requires an action to sample input to create the range.
	Takes a partitioner and an implicit ordering.			Slow with key skew

<code>repartitionAndSortWithinPartitions</code>	Partitions the RDD according to the partitioner and then sorts all the records on each partition according to the implicit ordering.	Key must have implicit ordering.	Depends on the partitioner. See <code>partitionBy</code> .	partitioner creates str and because sorting ea more expensive. How much faster than <code>partitionBy</code> and <code>t</code> within each key using <code>mapPartitions</code> be is pushed into the shuf “Secondary Sort and <code>repartitionAndSortWit</code>
<code>filterByRange</code>	Takes a lower and an upper bound for the keys and returns an RDD of just the records whose key falls in that range.	See above.	Almost never.	When the RDD has al partitioned by a range is cheaper than a gene because it scans only t whose keys are in the Otherwise performanc as the generic <code>filte</code>

Sorting by Two Keys with `SortByKey`

Spark’s `sortByKey` does allow sorting by tuples of keys for tuples with two elements. Thus, it is relatively easy to sort by two values in Spark by making use of Scala’s implicit tuple ordering, then sort by a composite key described by `(Key1, Key2)`. For example, suppose that rather than finding the `n`th item, Goldilocks just wanted us to create a directory of `(columnIndex, value)` pairs sorted first by column index then by value. Assuming an RDD `indexValuePairs` of type `RDD[(Int, Double)]`, we could solve this problem with:

```
indexValuePairs.map((_, null)).sortByKey()
```

Then `sortByKey` will use the implicit ordering on an `(Int, Double)` tuple, which simply compares the first value and then compares the second.

NOTE

`sortByKey` does not support implicit ordering on product types other than `Tuple2`.

Secondary Sort and repartitionAndSortWithinPartitions

Sorting in Spark could be implemented by partitioning an RDD with a `RangePartitioner` and then sorting within each partition using `mapPartitions`, much as we did in [Example 6-4](#). However, this approach to sorting is slower than Spark's actual implementation of `sortByKey`. Instead of partitioning and then sorting, Spark leverages a technique called *secondary sort*, which pushes some of the work of sorting on the individual machine into the shuffle stage.

NOTE

Secondary sort is a performant way of ordering data both amongst machines and within a single machine. The term comes from the MapReduce paradigm and describes a technique by which the programmer maps with one function, but defines a different order for the elements to be used in the reduce call. The effect of this in Spark is that some of the sorting work that must be done locally can be accomplished during the shuffle stage rather than in the next stage, after the shuffle has completed.

Spark has a built-in function to perform secondary sort called `repartitionAndSortWithinPartitions`. The `repartitionAndSortWithinPartitions` function is a wide transformation that takes a partitioner—defined on the argument RDD—and an implicit ordering, which must be defined on the keys of the RDD. The function partitions the data according to the partitioner argument and then sorts the records on each partition according to the ordering.

NOTE

We do not need to directly pass an implicit ordering to the `repartitionAndSortWithinPartitions` function. If the function is called on an RDD whose keys have an ordering, Spark can infer that ordering and will sort accordingly. To use `repartitionAndSortWithinPartitions` to order on types that either do not have an implicit ordering or have an ordering other than the one we want, we need to define the implicit ordering on the keys of the RDD in our program before calling the `repartitionAndSortWithinPartitions` function. See [Example 6-12](#).

If we look up the implementation of `sortByKey`, we can see that it calls the `repartitionAndSortWithinPartitions` function with a `RangePartitioner` and uses implicit ordering defined on the keys. As we discussed in “[Range Partitioning](#)”, the `RangePartitioner` will sample the data and assign a range of values for each partition based on the inferred distribution of the keys (for example, keys with values between 0 and 10 shall be placed on partition index two). Then `repartitionAndSortWithinPartitions` will sort the values on each partition (each range of data) and thus the entire result will be sorted by key. The secondary sort paradigm and the `repartitionAndSortWithinPartitions` can be used not only to